

Systèmes d'exploitation I

Module: M3 - INF-S3 & IAP-S3

Prof. R. HANNANE

r.hannane@uca.ma

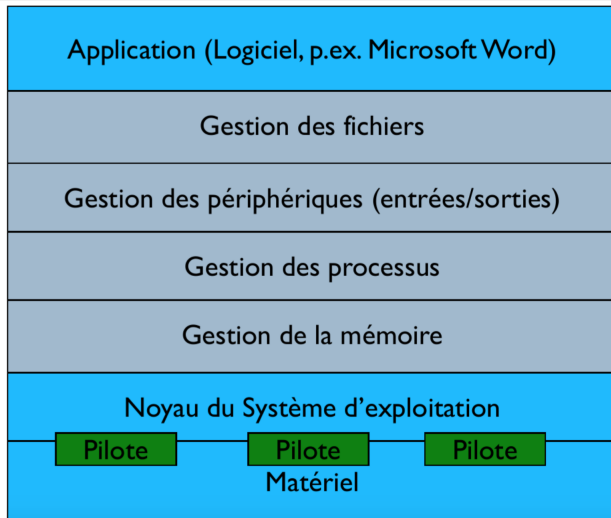
Faculté des Sciences Senglalia, Marrakech
Université Cadi Ayyad



Notions de base en systèmes d'exploitation (suite)

Les fonctions d'un système d'exploitation

Modèle en couches



Les fonctions d'un système d'exploitation

Modèle en couches

Gestion de la mémoire

- Le système d'exploitation est chargé de **gérer l'espace mémoire alloué à chaque application et à chaque usager** (allocation, libération d'espace)
- En cas d'insuffisance de mémoire physique, le système d'exploitation peut **créer une zone mémoire sur le disque dur**, appelée "**mémoire virtuelle**"
- La mémoire virtuelle **permet de faire fonctionner des applications nécessitant plus de mémoire qu'il n'y a de mémoire vive disponible sur le système**. En contre partie, **cette mémoire est beaucoup plus lente**
- Quelques tactiques appliquées par le système d'exploitation pour gérer la mémoire :
 - ▶ Partitions
 - ▶ Swapping
 - ▶ Pagination
 - ▶ Mémoire virtuelle

Les fonctions d'un système d'exploitation

Modèle en couches

Gestion des processus

- **Gérer l'exécution des processus en leur affectant les ressources nécessaires à leur bon fonctionnement**
- **Assurer la communication entre les processus** ainsi que leur synchronisation
- **Gère également les conflits dus à la concurrence et leurs solutions**
- **La principale tâche du système d'exploitation concerne l'allocation du processeur aux processus:**
 - ▶ Décider quel processus s'exécute à un moment donné
 - ▶ A quel moment interrompre le processus
 - ▶ Quel sera le suivant
 - ▶ De quoi il a besoin comme ressources pour son exécution
- **Le système d'exploitation doit gérer deux types de processus : les siens et ceux des utilisateurs**

Les fonctions d'un système d'exploitation

Modèle en couches

Gestion des entrées/sorties (E/S)

- Les **E/S** correspondent aux **mécanismes qu'utilisent les processus pour communiquer avec l'extérieur**
- Les **périphériques de sortie** (moniteur, imprimante, haut-parleurs, graveur de CD,...) traduisent les informations reçues par les **périphériques d'entrée**, puis transmises au microprocesseur
- Les **périphériques d'entrée** (clavier, souris, lecteur de CD/DVDRom, scanner, micro, webcam,...) analysent les informations fournies et les transmettent au microprocesseur
- L'existence d'E/S dans un programme introduit de nombreux problèmes dus à :
 - ▶ La **différence de vitesse** entre les périphériques et l'unité centrale
 - ▶ Le **risque d'erreur de programmation** qui peut entraîner le blocage de l'unité centrale
- Le système d'exploitation permet:
 - ▶ Unifier et de contrôler l'accès des programmes aux ressources matérielle par l'intermédiaire des pilotes
 - ▶ Éviter de monopoliser l'unité centrale pendant le déroulement de l'E/S
 - ▶ Assurer un partage efficace des ressources entre les programmes

Les fonctions d'un système d'exploitation

Modèle en couches

Gestion des fichiers

- Le **système de fichier** offre à l'utilisateur une vision homogène et structurée des données et des ressources :
 - ▶ Disques
 - ▶ Mémoires
 - ▶ Périphériques
- Les données sont **stockés sous forme de fichiers** et **organisés**, en général, en **une structure arborescente**
- Le système de fichiers doit offrir des primitives afin de créer, modifier, lire, supprimer ces fichiers
- Il doit également gérer l'espace occupé par les fichiers ainsi que l'espace libre
- Il doit prendre en charge le partage et la protection par les droits d'accès des fichiers dans un environnement multi-utilisateurs

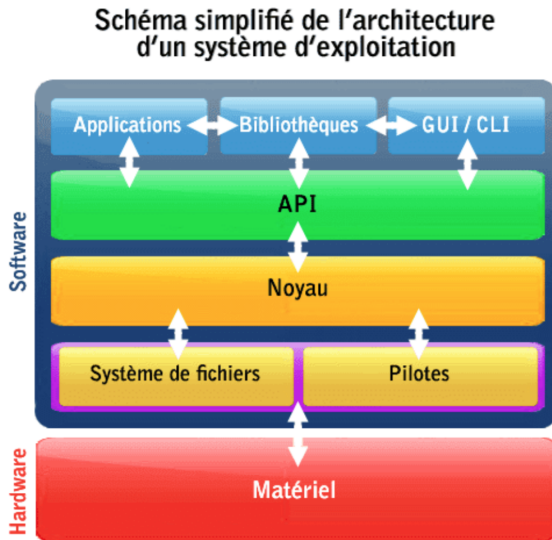
Les fonctions d'un système d'exploitation

Modèle en couches

Gestion de l'exécution des applications

- Le système d'exploitation est chargé de la **bonne exécution des applications** en leur affectant les ressources nécessaires à leur bon fonctionnement
- Il permet à ce titre de "tuer" une application ne répondant plus correctement

Eléments de base d'un système d'exploitation



Eléments de base d'un système d'exploitation

Le noyau (kernel en anglais)

- La **partie fondamentale de système d'exploitation**
- **Assure la communication logiciel / matériel**
- **Gestion des logiciels** : processus, lancement des programmes, ordonnancement, accès concurrents aux ressources (fichiers, imprimantes, ...)
- **Gestion du matériel** : processeur, périphériques au moyen de pilotes

Le système de fichiers

- **Façon d'organiser le stockage des données dans la mémoire secondaire et auxiliaires**
- Le système gère la **création**, la **destruction**, la **lecture** et l'**écriture** (modification) des fichiers, ainsi que les **droits d'accès**

Eléments de base d'un système d'exploitation

Les pilotes (ou drivers)

- Permettent **d'accéder** et de **contrôler le matériel**
- Généralement fournis par les constructeurs

L'API - (Application Programming Interface)

- Une **interface de programmation**
- Un **ensemble de outils de base et de protocoles** qui facilite la **création et l'intégration de logiciels d'applications**
- **L'objectif** est de **fournir une porte d'accès à une fonctionnalité** en **cachant les détails de la mise en œuvre**

Les bibliothèques

- **Morceaux de programmes**, qui peuvent être déjà compilés, à disposition du système d'exploitation et des programmes applicatifs
- Plusieurs formats : **.dll** (Windows), **.ocx**, **.dylib**, **.so**, **.a** (Unix/Linux)

Eléments de base d'un système d'exploitation

L'interface homme-machine

- Permet de faire communiquer l'utilisateur et la machine
- Deux types :
 - ▶ **GUI (Graphical User Interface)** : à travers des interfaces graphiques bien définies
 - ▶ **CLI (Command Line Interface)** : langage de commande interprété (Ex : Shell)

Applications/Programmes systèmes

- Composée des applications et des logiciels, entièrement indépendante du système d'exploitation
- Particulièrement, les programmes systèmes **souvent installés ensembles avec le système d'exploitation**
 - ▶ Ex : calculatrice, éditeur de texte, navigateur web
 - ▶ Gestion de fichiers : ls, mv, mkdir, rm...
 - ▶ Support de langages de programmation : cc, java...

Les principaux systèmes d'exploitation

- Systèmes **temps réel** : **Contrainte temporelle sur les exécutions**
 - ▶ Exemple : [Linux-rt](#), [Adeos](#), [QNX](#)
- Systèmes **embarqués** : **Prévus pour fonctionner sur des machines de petite taille**
 - ▶ Exemple : [PalmOS](#), [Windows CE](#), [Android](#), [Symbian](#)
- Systèmes **portables** : Smartphones, tablettes. Ils ont la **contrainte de mémoire limitée, processeurs lents, et de consommation**
 - ▶ Exemple : [Android](#), [iOS](#), [PureOS](#)
- Systèmes pour les **serveurs** : **Stabilité / fiabilité**
 - ▶ Exemple : [Unix](#), [Linux](#), [Windows Server](#)
- Systèmes **personnels** : **Pratique et facile à utiliser**
 - ▶ Exemple : [Windows](#), [Linux](#), [MacOS](#)

Unix/Linux

Bref historique de Unix/Linux (1/3)

- Le système **Unics** (Uniplexed Information and Computing Service) a été créé aux laboratoires *AT&T* de *Bell*, **en 1969** sous l'impulsion de deux hommes **Ken Thompson** et **Dennis Ritchie**
- Il était **destiné à fournir aux programmeurs maison un environnement de développement auquel ils avaient goûté avec Multics** (→ **Unics** a été une version réduite de **Multics** (*Multiplexed Information and Computing Service*))
- Peu après, le nom **Unics** a été remplacé par **Unix**, et à partir de ce moment le système Unix a commencé un long chemin de développement technique
- Suite au développement par **Dennis Ritchie** du langage C, une **nouvelle version d'Unix est réécrite** (en grande partie en C) **en 1973**
⇒ **Le système Unix a connu un véritable succès**

Bref historique de Unix/Linux (2/3)

- Pour la première fois, un système d'exploitation est écrit en langage non assembleur, et devient donc portable
 - L'Université de *Californie* à *Berkeley* a obtenu une copie de la version 6 du système Unix $\Rightarrow$ AT&T et *Berkeley* **ont séparément apporté de nombreuses modifications et améliorations au système Unix**
 - Le projet **Posix** (Portable Operating System UnIX) **de normalisation du système Unix a permis de développer un standard qui définit un ensemble de procédures**
- $\Rightarrow$ Tout système Unix conforme à Posix fournit ces procédures ou appels système standards (Ces procédures constituent la *bibliothèque standard d'Unix*)

Bref historique de Unix/Linux (3/3)

- Tout logiciel écrit en n'utilisant uniquement les procédures de la norme Posix devrait fonctionner sur tous les systèmes Unix conformes à cette norme
- Une version gratuite d'Unix porte le nom de Linux (code source disponible)
- La version Linux a été créée par Linus Torvalds en 1991, et par la suite, un grand nombre de programmeurs ont contribué à son développement accéléré

Les facteurs de succès de Linux

- Le succès de Linux tient à plusieurs facteurs :
 - 1 Le code source du système, ainsi que le noyau, les programmes utilisateurs, les outils de développement **sont librement distribuables** (*licence GPL, ou GNU*)
 - 2 Linux est **compatible** avec un certain nombre de standards **Unix** au niveau du code source, incluant les spécifications POSIX, system V et BSD
 - 3 Un très grand nombre d'applications Unix gratuites disponibles sur Internet se compilent sous Linux sans aucune modification
 - 4 Le système Linux a été **développé pour les processeurs Intel** et utilise toutes les fonctionnalités de ce processeur

Caractéristiques de Linux (1/3)

Linux est un système d'exploitation :

- 1 **Gratuitement téléchargeable**
- 2 **Multiutilisateurs, multitâches:** plusieurs utilisateurs peuvent accéder simultanément au système et exécuter un ou plusieurs programmes
- 3 **Populaire:** fonctionnant sur des machines très économiques (exemple Intel 386) et reconnaisse de plus en plus de périphériques de tout genre
- 4 **Multiplateformes:** s'installe sur une machine disposant déjà d'un autre système d'exploitation de type *Windows* et il peut même reconnaître ses fichiers et les accéder

Caractéristiques de Linux (2/3)

- 5 **Interopérable:** prend en charge de nombreux système de fichiers outre que le natif *ext2fs* (**VFAT** pour la FAT 32 de Windows 95/98; **NTFS** pour Windows NT; **SO9660** pour les CDROM; **Sys V** pour Unix Système V; **MS-DOS** pour la FAT 16 de DOS et de Windows 95,...)
- 6 **Doté d'une impressionnante logithèque:**
 - ▶ **bureautique** (*Star Office* ou *Open Office*),
 - ▶ **serveur web** (*Apache*),
 - ▶ **serveur de données** (*MySQL*, *Postgree...*),
 - ▶ **serveur d'application web** (*Zope*),
 - ▶ **serveur d'impression** (*Samba*),
 - ▶ **Firewall** (filtrage de paquet par *IPTables*)
- 7 **Largement documenté:** (exemple projet LDP (Linux Documentation Project))

Caractéristiques de Linux (3/3)

- 8 **Communiquant:** prend en charge les protocoles *TCP/IP, AppleTalk, X-Window, NFS, PPP, ATM...*
- 9 **Système modulaire:** facile à maintenir et à faire évoluer car il est conçu en couches
- 10 **Orienté temps partagé:** les ressources du processeur et du système sont réparties entre les utilisateurs
- 11 **Portable:** disponibilité des codes sources des logiciels qui le composent
- 12 C'est un SE dont les systèmes de fichiers sont hiérarchisés en arbre: plusieurs systèmes de fichiers peuvent être rattachés au système de fichiers principal $\Rightarrow$ ce qui rend le système indépendant du matériel et en assure la portabilité

Les distributions Linux

- **Linux** est le plus souvent diffusé sous forme d'une distribution
 - Une **distribution** est un ensemble cohérent de logiciels assemblé sous le noyau **Linux/GNU** et formant un système d'exploitation complet
 - Il existe de nombreuses distributions Linux, et chacune possède un fonctionnement qui lui est propre, et répondant à des objectifs variés
 - Chacune des distributions a ses avantages et ses inconvénients
- ⇒ **Débuter avec Linux**, c'est surtout choisir une distribution qui corresponde avec les usages qu'on attend du système

Les distributions Linux

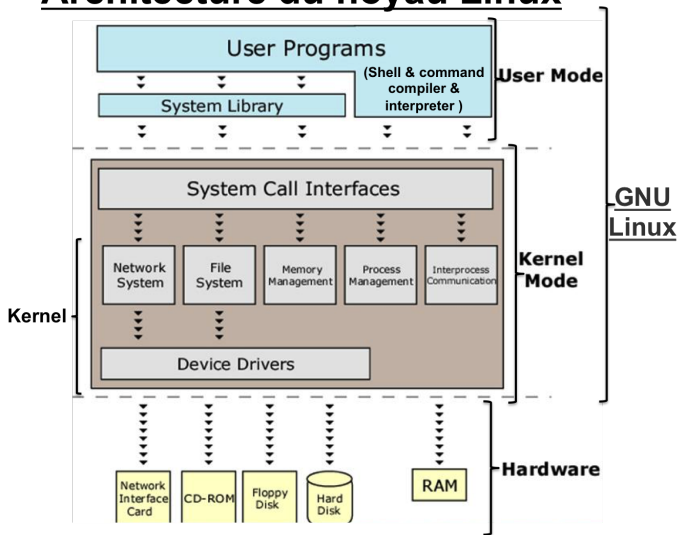
- Les **principales distributions de Linux** sur le marché:



...

Architecture logicielle de Linux

Architecture du noyau Linux



Structure de l'architecture logicielle de Linux

- Le système GNU/Linux est structuré essentiellement de :
 - ▶ **Espace Noyau** avec des interfaces et drivers de périphérique
 - ▶ **Espace utilisateur** avec Programme système (shell, compilateur, commande...)
 - L'interface entre les programmes des utilisateurs et le noyau est définie par un ensemble de procédures et de fonctions, soit directement au niveau du noyau, soit par l'intermédiaire de bibliothèques
 - Le noyau assure la gestion des ressources physiques (processeur, mémoires, périphériques,...) et logicielles (processus, fichiers...)
- ⇒ Le noyau réside entre interface appels systèmes et hardware
→ L'utilisateur n'accède pas directement au noyau mais à un interpréteur de commandes

Les appels système

- Certains **traitements ne peuvent pas être exécutés et contrôlés directement par un programme utilisateur** car le système ne le permet pas
⇒ Le **programmeur doit faire explicitement appel aux services du système d'exploitation pour accéder à certaines ressources**
- À ces fins, le **système d'exploitation propose une interface de programmation**, c'est-à-dire qu'il **permet d'accéder à un certain nombre de fonctionnalités** qu'il exécute pour l'utilisateur
- Les **appels système** sont les **interfaces proposées par le système d'exploitation pour accéder aux différentes ressources de la machine**

Les appels système

Leurs but

- Un **appel système** a pour but de :
 - ▶ **Changer de mode d'exécution** pour passer du mode utilisateur au mode maître
 - ▶ **Récupérer les paramètres et vérifier la validité de l'appel**
 - ▶ **Lancer l'exécution de la fonction demandée**
 - ▶ **Récupérer la (les) valeur(s) de retour**
 - ▶ **Retourner au programme appelant** avec retour au mode utilisateur
 - **Pour chaque composant de l'ordinateur, le système d'exploitation propose des appels système**
- ⇒ **Seuls ces appels systèmes peuvent être utilisés, le reste des primitives du système est protégé et leur usage direct est interdit aux utilisateurs**

Les appels système

Types d'appels systèmes

- Les appels système sont généralement **classés en 5 catégories** :
 - ▶ **Contrôle de processus**
 - charger, exécuter, créer, terminer des processus, obtenir, signaler, des événements, libérer de la mémoire, etc
 - ▶ **Manipulation des fichiers**
 - créer, supprimer, ouvrir, fermer, lire, écrire, repositionner, etc
 - ▶ **Gestion de périphériques**
 - demander, libérer, obtenir, attacher, etc
 - ▶ **Entretien d'information**
 - obtenir, définir l'heure ou la date, définir les données du système, etc
 - ▶ **Communications**
 - créer, supprimer des connexions de communication, envoyer, recevoir de messages, transférer des informations sur les états, etc

Les appels système

Exemple d'utilisation

- Il est nécessaire de **faire appel au système d'exploitation pour créer un fichier** sur le disque dur et cela via un appel système comme **createfile** sous Windows 2000
- Il est logique d'interdire au programme utilisateur d'effectuer lui-même la création pour les raisons suivantes:
 - ▶ Le **système d'exploitation** doit vérifier si l'utilisateur dispose du droit d'écriture dans le répertoire, autrement il refusera de créer le fichier
 - ▶ Il doit également vérifier si un fichier du même nom n'existe pas déjà
 - ▶ La création du fichier nécessite l'attribution de quelques secteurs du disque dur pour l'enregistrement des données du fichier → *L'utilisateur n'a aucune connaissance des zones libres du disque*

Les appels système

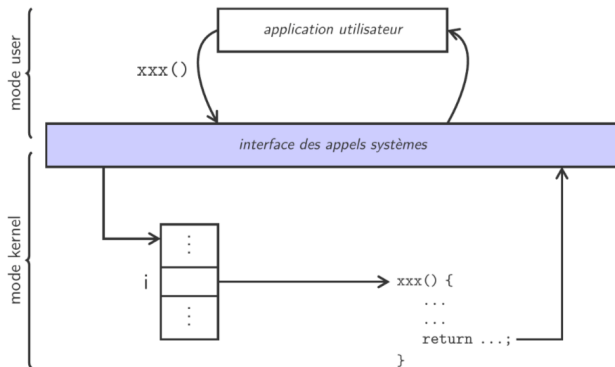
Exécution d'un appel système

- Lorsqu'un **programme** effectue un **appel système**, son exécution en mode utilisateur est interrompue et le **système** prend le contrôle en mode superviseur
- ⇒ **Un appel système est exécuté en mode noyau même si le programme ayant demandé son exécution est exécuté en mode utilisateur**
- Gestion via une **interface des appels systèmes**:
 - ▶ Fais le relais en interceptant les appels systèmes dans l'API
 - ▶ Utilise une **table associative** numérotant les appels systèmes
- Le **programme utilisateur** ignore les détails de l'appel système
 - ▶ Il doit respecter l'API et passer les bons paramètres
 - ▶ Il doit comprendre la valeur de retour

Les appels système

Exécution d'un appel système

- Sur la majorité des systèmes d'exploitation, les appels système peuvent être utilisés comme de simples fonctions écrites en C, leur exécution donne en retour un résultat. Si l'appel échoue, le résultat vaut -1



Exécution d'un appel système

Exemple d'algorithme d'un appel système pour la copie d'un fichier

❶ Récupérer le nom du fichier source

- ▶ Afficher une phrase sur la sortie standard
- ▶ Lire l'entrée standard

❷ Récupérer le nom du fichier destination

- ▶ Afficher une phrase sur la sortie standard
- ▶ Lire l'entrée standard

❸ Ouvrir le fichier source, créer et ouvrir le fichier destination

- ▶ **Tant que la lecture n'échoue pas**
 - Lire un caractère depuis la source
 - Écrire le caractère dans la destination

❹ Fermer les fichiers source et destination

Exécution d'un appel système

Exemple de code d'un appel système pour la copie d'un fichier

Version simplifiée de la copie d'un fichier

```
void copy ( char * src , char * dst )
{
    char buffer [ BUFFSIZE ];
    int srcFile = open ( src , O_RDONLY ) ;
    int dstFile = open ( dst , O_WRONLY | O_CREAT , 0666 ) ;
    if ( srcFile != -1 && dstFile != -1)
    {
        int r ;
        while ( ( r = read ( srcFile , buffer , BUFFSIZE ) ) )
        {
            write ( dstFile , buffer , r ) ;
        }
        close ( srcFile ) ;
        close ( dstFile ) ;
    }
}
```